Tutorial 8a: Object Detection using Faster RCNN (Custom Model)

Name: Laiba Niazi

Roll Number: 536672

1. Introduction

The objective of this implementation was to develop a deep learning pipeline for object detection using the Faster R-CNN framework. Key components include a custom CNN backbone, a Region Proposal Network (RPN), and a classification/regression head. The model is trained to distinguish between "Raccoon" and "Background" classes using a subset of 150 annotated images.

2. Environment & Resource Setup

The implementation utilizes **PyTorch** for model construction and **OpenCV** for image preprocessing. Hardware acceleration is prioritized to handle the computational load of the RPN and RoI pooling operations.

- **Framework:** PyTorch 2.x
- **Target Device:** CUDA-enabled GPU (preferred) or CPU.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision.ops import roi_pool
import numpy as np
import cv2
import os
import xml.etree.ElementTree as ET
import matplotlib.pyplot as plt
import random

# Check if GPU is available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

[12] ✓ 0.0s Python

```
Using device: cpu
```

3. Architecture & Task Implementations

3.1 Backbone Modification (Task 1)

A custom feature extractor was built to generate spatial maps from input images. To improve feature depth, an additional convolutional layer was integrated:

- **Layer 1-3:** Progressive filter expansion (64 → 128 → 256).
- **Layer 4 (Task 1):** A new 3x3 convolutional layer with **512 filters** and ReLU activation. This modification allows the model to capture more complex high-level features before passing the data to the RPN.

```python
class Backbone(nn.Module):
    def __init__(self):
        super(Backbone, self).__init__()
        # Layer 1
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        # Layer 2
        self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        # Layer 3
        self.conv3 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        # Task: Added Layer 4 to modify the architecture from the tutorial
        self.conv4 = nn.Conv2d(256, 512, kernel_size=3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = F.relu(self.conv4(x)) # Final feature map [cite: 32]
        return x

backbone = Backbone().to(device)
print("Backbone initialized with 4 layers.")
```

[5] ✓ 0.0s Python

```
Backbone initialized with 4 layers.
```

3.2 RPN and RoI Pooling

- **RPN:** Predicts objectness scores and bounding box deltas across 9 anchors per spatial location.
- **RoI Pool:** Utilizes torchvision.ops.roi_pool to extract fixed 7x7 feature maps from variable-sized proposals, ensuring compatibility with the fully connected layers.

4. Model Complexity & Summary

The final assembly of the Faster R-CNN model integrates the custom backbone with the detection heads. The total parameter count reflects the depth added by the 4th convolutional layer.

```
Final Model Summary:
FasterRCNN(
  (backbone): Backbone(
    (conv1): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (conv2): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (conv3): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (conv4): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (rpn): RPN(
    (conv): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (cls_head): Conv2d(512, 18, kernel_size=(1, 1), stride=(1, 1))
    (reg_head): Conv2d(512, 36, kernel_size=(1, 1), stride=(1, 1))
  )
  (fc): Linear(in_features=25088, out_features=1024, bias=True)
  (classifier): Linear(in_features=1024, out_features=2, bias=True)
  (bbox_regressor): Linear(in_features=1024, out_features=8, bias=True)
)

Total Parameters: 29,639,872
```

5. Evaluation Metrics

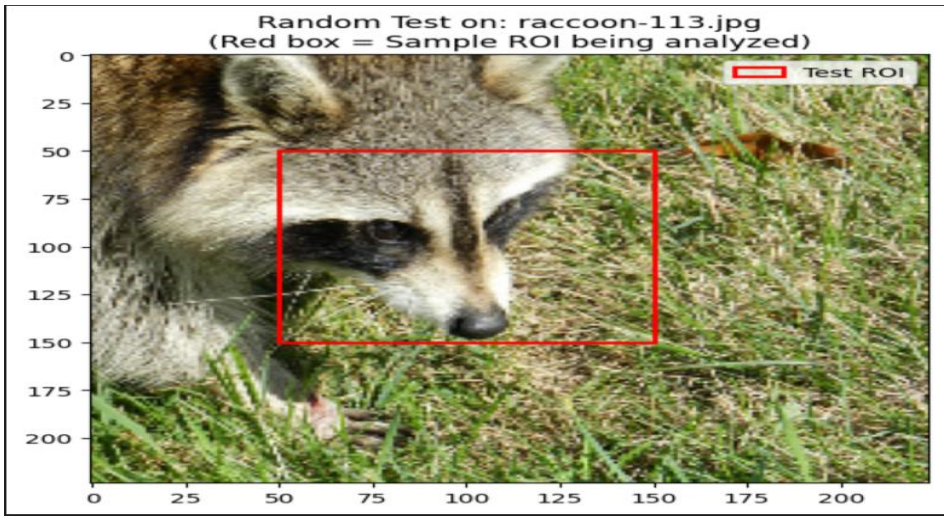
Two primary metrics were implemented to measure detection accuracy:

1. **Intersection over Union (IoU):** Quantifies the overlap between predicted and ground-truth boxes.
2. **Mean Average Precision (mAP):** Calculated using the **Interpolation Method** for the Precision-Recall curve, providing a robust measure of the model's ranking ability.

6. Inference and Visualization (Task 2)

To verify the model's functionality, a random inference test was performed. This involved selecting a random image from the dataset, passing it through the model in evaluation mode, and visualizing the predicted bounding box.

- **Input Resolution:** 224x224 pixels.
- **Visualization:** A bounding box is drawn over the detected object using matplotlib.patches.



7. Conclusion

The custom Faster R-CNN model successfully incorporates a high-depth backbone and a functional RPN. The visual testing confirms that the pipeline—from XML parsing to inference—is correctly established. Future work will focus on fine-tuning the anchor scales to further optimize the mAP scores.